Studente1 (30L)

- **Quali sono gli operatori utilizzabili per definire le espressioni regolari in Flex?**
  *(Esempio: ¦, *, +, ? e così via.)*
- **Spiega la sintassi delle espressioni regolari in Flex.**
  (Ad esempio, come si definiscono gruppi, classi di caratteri, o operatori come il punto per "qualsiasi carattere".)
- **Quante espressioni regolari è possibile specificare in un file Flex?**
  (Limiti teorici o pratici nel numero di regole definibili.)
- **Qual è la struttura di un file Flex?**
  - Quali sono le principali sezioni e la loro funzione?
  - Qual è la differenza tra due regole diverse nel file Flex?
- **Cos'è e come funziona la regola degli implicatori nel linguaggio C?**
  (Ad esempio, la gestione automatica dei tipi nei contesti specifici.)
- **Quali sono le priorità tra le espressioni regolari in Flex?**
  (Come vengono risolti conflitti tra più regex che potrebbero corrispondere allo stesso input.)
- **Dove vengono definite le parole chiave in un file Flex?**
  (Ad esempio, nell'intestazione o nella sezione delle dichiarazioni iniziali.)
- **Scrivi un esempio di definizione in Flex in cui:**
  - `"forza"` è un identificatore generico, e
  - `"for"` è una keyword con trattamento speciale.
    (Mostra come differenziare tra keyword e identificatori generici.)

Studente2 (28)

- **Quali sono i diversi tipi di Intermediate Representation (IR)?**
  - IR lineari
  - IR strutturali
  - IR ibride
- **Quando finisce un Basic Block (BB)?**
  - Esempi di BB di entrata e BB di uscita.
  - Regole che determinano la fine di un BB (ad esempio, istruzioni di salto o terminatori).
- **Quali sono i criteri che definiscono la fine di un Basic Block in LLVM?**
  - Presenza di istruzioni di salto.
  - Una `return` rappresenta il terminatore del Control Flow Graph (CFG).
- **Cosa troviamo all'interno di un modulo LLVM?**
  - Funzioni, variabili globali, e metadati.
- **Cosa rappresenta il Control Flow Graph (CFG) in LLVM?**
  - Una funzione.
  - Le analisi basate sul CFG sono dette "intraprocedurali" perché si concentrano solo sugli elementi interni alla funzione/procedura.
- **Come si chiamano le analisi che si basano sul CFG?**
  - Analisi forward.
  - Analisi backward.
- **Qual è il limite oltre il quale il CFG non può andare?**
  - Il CFG non può rappresentare dipendenze tra funzioni diverse (si limita a una singola funzione).
- **Quali sono le caratteristiche principali del CFG in LLVM?**
  - Rappresentazione di flussi di controllo.
  - Ogni nodo del grafico rappresenta un Basic Block, e gli archi rappresentano le transizioni tra i blocchi.
- **Qual è la differenza tra codice orientato a registri (reg2reg) e codice orientato a memoria (load & store)?**

- **Codice orientato a registri:** le operazioni lavorano direttamente con i registri.
- **Codice orientato a memoria:** le operazioni richiedono caricamenti (LOAD) e salvataggi (STORE) espliciti da e verso la memoria.

- **Qual è lo scopo delle funzioni phi in LLVM?**
  - Implementare l'assegnazione condizionale all'interno di una rappresentazione in Static Single Assignment (SSA).
  - Permettere la convergenza di valori da percorsi differenti in un Basic Block.
- **Fornisci un esempio in cui viene violata la Static Single Assignment (SSA).**
  - Quando una variabile viene assegnata più volte senza essere rinominata, violando il principio di unicità delle definizioni in SSA.

Studente3 (30)

- **Qual è lo scopo dell'analisi sintattica nei compilatori?**
  - Verificare che il codice sorgente rispetti la grammatica del linguaggio.
  - Costruire una rappresentazione strutturale del codice, tipicamente un albero di derivazione o un albero sintattico (AST).
- **Cos'è un parser top-down predittivo?**
  - Un parser che analizza il codice partendo dal simbolo iniziale della grammatica e tenta di predire quali regole applicare basandosi sul prossimo simbolo in ingresso.
  - Perché esista, la grammatica deve essere LL(1).
- **Come si elimina la ricorsione sinistra?**
- **Cosa significa per una grammatica avere la ricorsione sinistra?**
- **Cosa significa LL(1)?**
  - **L:** analisi Left-to-right (da sinistra a destra).
  - **L:** costruzione della derivazione Leftmost (la derivazione più a sinistra).
  - **1:** una sola lookahead token (un simbolo di anticipo) per decidere quale produzione utilizzare.
- **Cosa sono i set FIRST e FIRST+?**
  - **FIRST(X):** l'insieme dei terminali che possono iniziare una stringa derivabile da (X).
  - **FIRST+(X):** l'insieme dei terminali che possono iniziare una stringa derivabile da (X), tenendo conto anche del simbolo successivo per risolvere ambiguità.
- **Quando una grammatica è LL(1)?**
  - Una grammatica è LL(1) se, per ogni produzione, l'intersezione tra i set FIRST delle produzioni alternative di un simbolo non terminale è vuota, e se non c'è conflitto tra FIRST e FOLLOW per la gestione di (ϵ).
- **Che parser costruisce Bison?**
  - Bison costruisce un parser bottom-up, tipicamente un parser LR(1) o LALR(1).
- **Qual è la differenza tra parser top-down e bottom-up?**
  - **Top-down:** inizia dal simbolo iniziale e cerca di risalire alla stringa in ingresso.
  - **Bottom-up:** parte dalla stringa in ingresso e cerca di ridurla al simbolo iniziale, applicando le produzioni in ordine inverso.
- **Perché l'insieme dei linguaggi LR(1) è più grande di quello dei linguaggi LL(1)?**
  - I parser LR(1) possono gestire grammatiche più complesse, incluse quelle con ambiguità locali risolvibili mediante analisi bottom-up e lookahead.
- **Fornisci un esempio di conflitto shift-reduce.**
  - Si verifica quando il parser non sa se eseguire una riduzione con una regola esistente o fare uno shift per analizzare il prossimo simbolo in ingresso.

Studente4 (28)

- **Come funzionano i parser bottom-up?**
  - Partono dalla stringa in ingresso e cercano di ricostruire il simbolo iniziale della grammatica applicando le produzioni in ordine inverso.
  - Utilizzano una tabella con due operazioni principali:
    - **Shift:** aggiunge un simbolo allo stack e avanza nel leggere la stringa.
    - **Reduce:** sostituisce una sequenza di simboli nello stack con il non terminale corrispondente secondo una produzione.

- **Che cos'è una handle?**
- **Differenza tra parser top-down e bottom-up:**
    - **Top-down:** analizza da sinistra verso destra cercando di prevedere quale produzione applicare basandosi sul lookahead (FIRST).
    - **Bottom-up:** analizza da sinistra verso destra, ma costruisce la derivazione al contrario utilizzando lookahead basato su FOLLOW.
    - **Lookahead:**
        - Top-down: controlla il primo simbolo seguente nella stringa (FIRST).
        - Bottom-up: controlla il primo simbolo seguente il contesto della produzione (FOLLOW).
    - I parser bottom-up sono più potenti dei top-down perché possono gestire grammatiche più complesse.
- **Cosa succede in Bison quando una grammatica è ambigua?**
    - Bison costruisce una tabella composta da due sezioni:
        - **Action:** specifica se fare uno shift, una reduce o segnalare un errore.
        - **Goto:** indica quale stato raggiungere dopo una riduzione.
    - Se la grammatica è ambigua, si verificano conflitti (e.g., shift-reduce o reduce-reduce) che vengono segnalati nel report.
- **Come può fallire la costruzione di una tabella in Bison?**
    - Quando non è possibile risolvere i conflitti presenti nella grammatica (e.g., ambiguità non gestite).
    - Se ci sono errori nella definizione delle regole che impediscono la costruzione corretta delle tabelle Action e Goto.
- **Cosa possiamo trovare nel report di Bison quando si verifica un problema?**
    - Informazioni sui conflitti rilevati, tra cui:
        - Conflitti **shift-reduce** (ambiguità su quale operazione eseguire).
        - Conflitti **reduce-reduce** (ambiguità su quale regola ridurre).
    - Eventuali produzioni ambigue e stati problematici.
- **Cosa sono i conflitti reduce-reduce?**
    - Si verificano quando due o più produzioni sono applicabili per ridurre lo stesso simbolo nello stesso contesto, e il parser non sa quale scegliere.
- **Perché non puoi avere conflitti shift-shift?**
    - Perché il simbolo successivo da leggere è uno solo, quindi il parser non può eseguire due operazioni di shift contemporaneamente.
- **Esempio di ambiguità nei linguaggi imperativi:**
    - Ambiguità: non è chiaro a quale “if” appartiene l'“else” in presenza di “if-then-else”.
- **Tre modalità per rimuovere l'ambiguità:**
    - **Modificare la grammatica:** riscrivere le regole per evitare ambiguità (e.g., specificando esplicitamente come associare “else”).
    - **Far scegliere al compilatore:** utilizzare regole di precedenza e associatività per risolvere i conflitti.
    - **Modificare il linguaggio di programmazione:** introdurre nuove sintassi o restrizioni per eliminare situazioni ambigue (e.g., obbligare l'uso di parentesi o regole più stringenti).

Studente5 (28)

- **Come si implementa l’astrazione procedurale nel linguaggio macchina?**
    - Utilizzando l’**activation record** (detto anche record di attivazione), una struttura dati che contiene tutte le informazioni necessarie per eseguire una procedura.
    - L’activation record include:
        - Parametri della funzione.
        - Variabili locali.
        - Indirizzo di ritorno.
        - Eventuali informazioni di controllo e di gestione del contesto.
- **Dove si trova l’activation record?**
    - Tipicamente è allocato nello **stack**, ma può essere allocato nell’**heap** in base alle esigenze.
- **Quando l’activation record non può stare sullo stack?**
    - Quando il ciclo di vita della funzione chiamata supera quello della funzione chiamante.
        - Ad esempio:
            - **Stack:** utilizzato quando il chiamato termina prima o contemporaneamente al chiamante.
            - **Heap:** utilizzato quando si richiede che il chiamato viva più a lungo del chiamante (e.g., callback, coroutine, o funzioni che ritornano chiusure).

- **Come faccio a sapere dove si trova una variabile a tempo di esecuzione se non conosco i dettagli?**
    - Utilizzando l'**offset** relativo all'inizio dell'activation record.
    - Conoscendo l'offset, è possibile risalire all'indirizzo effettivo della variabile rispetto alla base dell'activation record.
- **Classificazione del codice:**
    - **Locale:** operazioni e informazioni che riguardano un singolo blocco di codice o funzione.
    - **Regionale:** ottimizzazioni e analisi che coinvolgono un insieme di blocchi di base vicini.
    - **Intraprocedurale:** analisi e ottimizzazioni limitate all'interno di una singola procedura o funzione (guardo il cfg di una funzione).
    - **Interprocedurale:** analisi e ottimizzazioni che attraversano i confini delle funzioni, considerando chiamate e dipendenze tra procedure.
- **Che cos'è un extended basic block?**
    - Un **extended basic block (EBB)** è una sequenza di istruzioni formata da un blocco base iniziale, seguito da altri blocchi che non sono punti di ingresso per salti esterni.
    - Tutte le istruzioni di un EBB sono eseguite sequenzialmente senza salti esterni, tranne all'uscita del blocco.
    - Permette di estendere le analisi e le ottimizzazioni oltre il singolo blocco base, mantenendo il controllo lineare sul flusso di esecuzione.

Studente6 (21)

- **Ottimizzazioni nelle analisi interprocedurali (inlining):**
    - L'**inlining** consiste nel sostituire una chiamata di funzione con il codice della funzione stessa.
    - Permette di evitare il costo di una chiamata di funzione e rende il codice più adatto a ottimizzazioni successive.
- **Benefici dell'inlining:**
    - Miglioramento delle prestazioni eliminando overhead delle chiamate di funzione.
    - Maggiore opportunità per ulteriori ottimizzazioni, come:
        - Propagazione costanti.
        - Rimozione del codice morto.
    - Riduzione della complessità del flusso di controllo a runtime.
    - **Svantaggi:** aumento delle dimensioni del codice (code bloat).
- **Struttura dell'activation record:**
    - Contiene informazioni necessarie per gestire l'esecuzione di una procedura. Include:
        - **Parametri:** passati alla funzione.
        - **Register save area:** spazio per salvare i registri temporanei usati durante la funzione.
        - **Return value:** spazio per il valore di ritorno della funzione.
        - **Return address:** indirizzo a cui tornare dopo aver completato la funzione.
        - **Addressability:** informazioni per risolvere riferimenti a variabili locali e parametri.
        - **ARP (Activation Record Pointer):** puntatore al record di attivazione corrente.
        - **Caller's ARP:** puntatore al record di attivazione del chiamante.
        - **Local variables:** variabili locali della funzione.
- **Cosa produce Flex?**
    - Flex genera un **lexer** (analizzatore lessicale) in linguaggio C.
    - Questo lexer legge un input testuale e riconosce i token basati sulle espressioni regolari specificate nel file di configurazione.
- **Come si passa da regex a NFA (metodo di Thompson)**
- **Qual è la grammatica delle espressioni regolari?**
    - Le espressioni regolari possono essere definite attraverso una grammatica formale:
        - **R → ε | a | (R1 R2) | (R1 | R2) | (R1*)**
        - Dove:
            - ε rappresenta la stringa vuota.
            - a è un simbolo del linguaggio.
            - (R1 R2) rappresenta la concatenazione di due regex.
            - (R1 | R2) rappresenta l'unione di due regex.
            - (R1*) rappresenta la chiusura di Kleene di una regex.
- **Come faccio a descrivere un linguaggio (con le grammatiche)?**
    - Utilizzando una **grammatica formale**:
        - Una grammatica è una quadrupla G = (N, T, P, S)
    - La grammatica genera tutte e sole le stringhe appartenenti al linguaggio.

- Si utilizza una combinazione di produzioni per costruire le stringhe del linguaggio partendo dal simbolo iniziale.

Studente7 (20)

- **Passaggi da regex a DFA minimizzato:**

  1. **Regex a NFA (metodo di Thompson)**
  2. **NFA a DFA (algoritmo subset construction)**
  3. **Minimizzazione del DFA**

- **Che cos'è formalmente un NFA (Non-deterministic Finite Automaton):**

  - Un NFA è una **quintupla** $A = (Q, \Sigma, \delta, q_0, F)$
  - L'NFA consente transizioni multiple o con ϵ, rendendolo non deterministico.

- **Come funziona la costruzione di Thompson**

- **Cosa significa per una variabile essere viva**

- **Quando una variabile è morta**

- **Come si calcola l'insieme delle variabili vive**

Studente8

- **NFA a DFA (algoritmo subset construction)**
- **Minimizzazione del DFA**
- Esempio bison visto a lezione: come calcolare i valori delle costanti numeriche (con i dollari)

Studente9 (30L)

- ancestor nell'AR
- scope statico

Studente10 (30L)

- cosa c'è nel front end del compilatore
- come viene implementato il checker